



Chapter 14 – Cross-Site Scripting (XSS)



Introduction to XSS (Cross-Site Scripting)

What is XSS?

Cross-Site Scripting (XSS) is a client-side web application vulnerability where an attacker injects malicious scripts (usually JavaScript) into a trusted website.

When another user visits the vulnerable page, the malicious script executes inside the victim's browser without their knowledge.



Important:

XSS attacks can only occur on websites that contain XSS vulnerabilities. If proper input validation and sanitization are implemented, XSS attacks will not work.



Simple Definition

Imagine a website has a comment box.

Normally users write:



Nice article!



Great tutorial!

But an attacker enters malicious code instead of normal text.

If the website stores and executes that code, the attacker can:



Steal cookies



Steal session tokens



Hijack accounts



Send unauthorized requests



Impersonate users



Redirect users to fake websites

Real-Life Example

Suppose a social media website has a comment section.

Normal user:

Nice Post!

Attacker:

```
<script>alert('Hacked')</script>
```

If the website displays the script without filtering it, every visitor's browser executes the code.

This is Cross-Site Scripting (XSS).

Why Is It Called Cross-Site Scripting?

Originally, attackers used scripts from one website to attack users on another website.

The name remained:

Cross-Site Scripting (XSS)

The abbreviation became **XSS** because CSS was already used for Cascading Style Sheets.

Important Concept from Lecture

For XSS to occur:

 There must be an input field

Examples:

 Comment Box

 Search Box

 Feedback Form

 Profile Bio

 Chat Message

 Contact Form

Without user-controllable input, XSS opportunities are much lower.

Understanding GET and POST Methods

Before understanding XSS, you must understand how data is sent to a server.

GET Method

Used to request information.

Data appears in URL.

Example

<https://example.com/profile?id=10>

The parameter is visible.

Common Uses

 Search Pages

 Viewing Profiles

 Reading Articles

POST Method

Used to send information to the server.

Data is sent in the request body.

Not directly visible in URL.

Common Uses

-  Login Forms
 -  Comment Submission
 -  Contact Forms
 -  Checkout Forms
-

Important XSS Point

Most Stored XSS attacks happen through POST requests because:

User submits data



Data is stored in database



Later displayed to other users

Understanding Cookies

What Are Cookies?

Cookies are small text files stored in a browser.

Websites use them to remember users.

Example

Instagram

Facebook

Gmail

Amazon

When you login:

Server sends a cookie.

Browser stores it.

Future requests contain that cookie.

You remain logged in.

Understanding Session Tokens

After login:

Server generates a unique session ID.

Example:

A7B3F92D8E

This identifies the user.

Why Attackers Want Session Tokens

If an attacker steals your session token:

- ✗ Username not required
- ✗ Password not required
- ✗ Sometimes even 2FA is bypassed

The website believes:

"This is the legitimate user."

This is called **Session Hijacking**.

Why Is XSS Dangerous?

XSS can:

-  Steal cookies
-  Steal session tokens
-  Hijack accounts

-  Send messages as victim
 -  Perform transactions
 -  Redirect users
 -  Download malware
 -  Launch phishing attacks
 -  Collect browser information
-

Input Sanitization (Very Important)

Your instructor mentioned an important defense.

What is Sanitization?

Sanitization means cleaning user input before processing it.

Example

User enters:

```
<script>
```

Website converts it into:

```
&lt;script&gt;
```

Now it becomes plain text instead of executable code.

Why Are < and > Important?

HTML tags require:

```
<tag>
```

Without:

```
<
```

```
>
```

scripts cannot be interpreted properly.

First Precaution Against XSS

- ✓ Replace dangerous characters

<

>

"

'

&

with safe encoded versions.

Types of XSS Attacks

There are three major types:

- 1 Reflected XSS
 - 2 Stored XSS
 - 3 DOM-Based XSS
-

1 Reflected XSS (Non-Persistent XSS)

Definition

Reflected XSS occurs when:

The malicious script is immediately reflected back in the server response.

It is NOT stored in the database.

How It Works

Step 1

Attacker creates malicious input.

Step 2

Input is sent to vulnerable website.

Step 3

Website reflects the input in response.

Step 4

Victim opens the malicious link.

Step 5

Script executes instantly.

Important Lecture Point

The script:

- ✗ Is not stored permanently
- ✗ Is not saved in database
- ✓ Executes immediately

Example

An attacker sends:

`https://site.com/search?q=<script>alert('XSS')</script>`

Victim clicks.

Browser executes script.

Real-Life Example

Suppose:

Website search box displays:

You searched for: XYZ

Attacker searches:

```
<script>alert('Hacked')</script>
```

Website prints it directly.

Alert executes.

Why Attackers Use Reflected XSS

Attackers can steal:

-  Cookies
-  Session IDs
-  User information

Prevention

-  Input Validation
-  Output Encoding
-  Content Security Policy
-  Avoid reflecting raw user input

2 Stored XSS (Persistent XSS)

Definition

Stored XSS happens when malicious code is stored permanently inside the website database.

How It Works

Step 1

Attacker submits malicious script.

Step 2

Website stores it.

Step 3

Another user opens page.

Step 4

Script automatically executes.

Example

Comment:

```
<script>alert('This Site Is Hacked')</script>
```

Stored in database.

Every visitor executes it.

Real-Life Example

Forum Website

Blog Comments

Product Reviews

User Profiles

Social Media Posts

Why Stored XSS Is Dangerous

One attack affects:

 User 1

 User 2

 User 3

 User 1000

Everyone who visits the page.

Prevention

- ☒ Sanitize before storing
 - ☒ Sanitize before displaying
 - ☒ Encode output
 - ☒ Validate all user input
-

3 DOM-Based XSS

Definition

DOM-Based XSS occurs entirely inside the browser.

The server may never see the malicious payload.

What is DOM?

DOM = Document Object Model

The browser creates a structure of the webpage.

JavaScript interacts with this structure.

Lecture Example

Suppose an online shoe store has:

 Size Selection

 Color Selection

 Product Filters

These are user-controlled inputs.

If JavaScript inserts user input into the page unsafely, attackers can inject scripts.

Example URL

`https://example.com/#<script>alert('Hacked')</script>`

The browser processes the URL fragment.

Unsafe JavaScript executes the payload.

Dangerous Functions

Avoid:

`document.write()`

`innerHTML`

with untrusted input.

Safe Alternatives

Use:

`textContent`

`createTextNode()`

Prevention

- ☒ Avoid innerHTML
- ☒ Use textContent
- ☒ Encode user data

- ✓ Validate URL parameters
-

XSS Testing Tools

DVWA

Damn Vulnerable Web Application

Purpose:

Practice XSS safely.

Features:

- ✓ Reflected XSS
 - ✓ Stored XSS
 - ✓ DOM XSS
-

bWAPP

Buggy Web Application

Used for:

-  Vulnerability Practice
 -  Security Testing
 -  Ethical Hacking Labs
-

Burp Suite

Purpose:

Intercept HTTP Requests

Modify Parameters

Test XSS Inputs

OWASP ZAP

Purpose:

Automatic Vulnerability Scanner

Detects:

- ✓ XSS
- ✓ SQL Injection
- ✓ Security Misconfigurations

Safe XSS Detection Examples

For learning and testing in authorized labs, common harmless payloads include:

```
<script>alert('XSS')</script>
```

```
<img src=x onerror=alert('XSS')>
```

If a vulnerable test application executes these alerts, it indicates that user-supplied input is being interpreted as code instead of plain text.

How to Defend Against XSS

Prevention Technique	Explanation
Input Validation	Accept only expected data
Output Encoding	Encode special characters
Sanitization	Remove dangerous code
HTTPOnly Cookies	Prevent JavaScript access
Secure Cookies	Protect cookies during transmission

Prevention Technique	Explanation
CSP	Restrict script execution
Framework Security Features	Use built-in protection
Regular Security Testing	Identify vulnerabilities early
WAF	Block malicious requests
Patch Management	Keep software updated

HTTPOnly Cookies

One of the strongest defenses.

Example:

Set-Cookie: SessionID=ABC123; HttpOnly

Now JavaScript cannot access the cookie.

This greatly reduces session theft risk.

Content Security Policy (CSP)

CSP controls:

- ✓ Which scripts may run
- ✓ Which domains are trusted
- ✓ Which resources can load

Example:

Content-Security-Policy:

script-src 'self'

Only scripts from the website itself are allowed.



Comparison of XSS Types

Feature	Reflected XSS		Stored XSS	DOM XSS
Stored on Server	✗ No	✓ Yes	✗ No	✗ No
Requires Victim Action	✓ Usually	✗ Often Automatic	✓ Usually	✓ Usually
Executes In Browser	✓ Yes	✓ Yes	✓ Yes	✓ Yes
Most Dangerous	Medium	High	Medium	Medium
Example	Search Box	Comment Box	URL Fragment	



Common XSS Attack Patterns (Defensive Study Guide)



1 Script Injection

What Happens?

An attacker attempts to inject JavaScript into a webpage through an input field.

Target Areas

- Comment boxes
- Search fields
- Feedback forms
- User profiles

Prevention



Input Validation



Output Encoding



Content Security Policy (CSP)



HTML Escaping

2 Cookie Theft Attempts

What Happens?

An attacker tries to access browser cookies containing session information.

Impact

- Session hijacking
- Account takeover

Prevention

- ✓ HttpOnly Cookies

Set-Cookie: sessionid=abc123; HttpOnly; Secure

- ✓ SameSite Cookies

Set-Cookie: sessionid=abc123; SameSite=Strict

- ✓ CSP
-

3 Session Token Theft

What Happens?

Attackers attempt to obtain authentication tokens stored in browsers.

Impact

- Login bypass
- Unauthorized access

Prevention

- ✓ Secure Session Management
 - ✓ Short Session Lifetime
 - ✓ MFA (Multi-Factor Authentication)
 - ✓ Token Rotation
-

4 Fake Login Form Injection

What Happens?

Attackers try to display fake login forms inside legitimate websites.

Impact

- Credential theft
- Phishing

Prevention

- ✓ CSP
- ✓ Frame Protection

X-Frame-Options: DENY

- ✓ User Awareness
-

5 Malicious Redirects

What Happens?

Users are redirected to attacker-controlled websites.

Impact

- Phishing
- Malware infection

Prevention

- ✓ Validate Redirect URLs
 - ✓ Allowlist Trusted Domains
-

6 DOM Manipulation Attacks

What Happens?

Client-side JavaScript modifies webpage content using unsafe methods.

Dangerous Functions

innerHTML
document.write()
outerHTML

Safer Alternatives

textContent
createTextNode()
setAttribute()

Prevention

- ✓ Sanitize Data Before Display
 - ✓ Use Secure Frameworks
-

7 Stored XSS

What Happens?

Malicious content is saved in a database and shown to future visitors.

Example Targets

- Blog comments
- Forums
- Reviews
- Chat applications

Prevention

- ✓ Validate Before Saving
 - ✓ Encode Before Displaying
 - ✓ Use Web Application Firewall (WAF)
-

8 Reflected XSS

What Happens?

Malicious input is immediately reflected back in a webpage response.

Common Locations

- Search pages
- Error messages
- URL parameters

Prevention

- ✓ Input Validation
 - ✓ Output Encoding
 - ✓ URL Parameter Sanitization
-

9 DOM-Based XSS

What Happens?

JavaScript running in the browser processes untrusted data and inserts it into the page.

Common Sources

- URL fragments
- Query strings
- Browser storage

Prevention

- ✓ Avoid Unsafe DOM APIs
 - ✓ Sanitize User-Controlled Data
-

Enterprise-Level XSS Prevention Checklist

Input Validation

Allow only expected input.

Example:

- Age → Numbers only
- Username → Letters and numbers only
- Email → Valid email format only

Output Encoding

Convert dangerous characters:

Character Encoded Form

<	<
>	>
"	"
'	'
&	&

Content Security Policy (CSP)

Example:

Content-Security-Policy: default-src 'self';

Benefits:

- Blocks unauthorized scripts
 - Prevents many XSS attacks
 - Restricts script execution
-

Secure Cookies

Set-Cookie: sessionid=xyz; Secure; HttpOnly; SameSite=Strict

Benefits:

- HTTPS only
 - JavaScript cannot read cookies
 - Reduces session theft risk
-

Use Modern Frameworks

Frameworks with built-in XSS protections:

- React
 - Angular
 - Vue.js
 - Django
 - Laravel
 - Spring Boot
-

Web Application Firewall (WAF)

Examples:

- [Cloudflare](#)
- [Imperva](#)
- [Akamai](#)

Benefits:

- Detects XSS patterns
 - Blocks malicious requests
 - Protects web applications
-